Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Assignment No. 1 — Sorting Algorithms

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Question 1: Merge Two Sorted Arrays

Problem Statement:

You are given two sorted arrays, A and B, where A has a large enough buffer at the end to hold B. Write a method to merge B into A in sorted order.

Proposed Solution:

Since array A already has empty space at the end to accommodate B, the arrays are merged starting from the back instead of the front. This avoids the need for any extra array and prevents overwriting of unprocessed elements in A.

Three pointers are maintained: one at the last valid element of A (i), one at the last element of B (j), and one at the last index of the combined array (k). At each step, the larger of arr[i] and brr[j] is placed at position k, and the corresponding pointer along with k is decremented.

Once all elements of B are placed (or A is exhausted first), any remaining elements of B are copied directly, since the remaining elements of A are already in their correct sorted position.

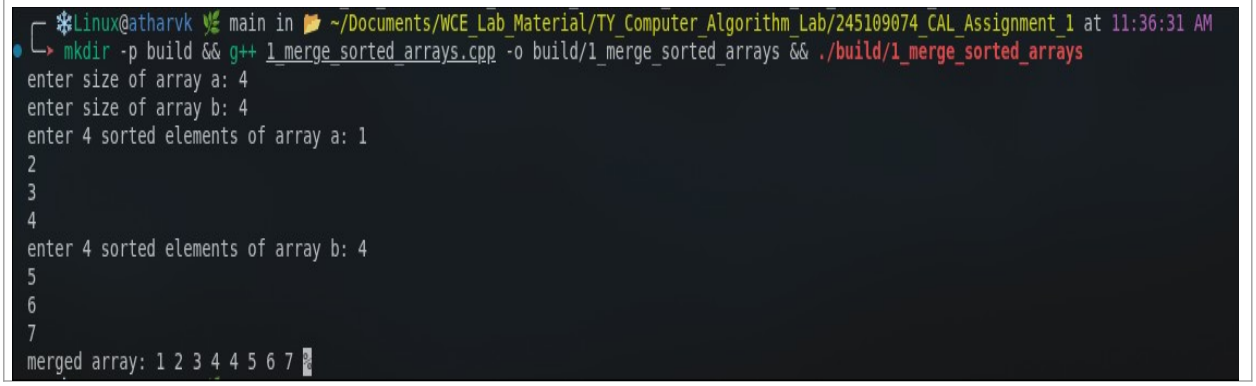
Time Complexity: O(n + m), where n and m are the sizes of arrays A and B respectively, since each element is visited and placed exactly once.

Space Complexity: O(1) additional space, as the merge is performed in-place within array A using the existing buffer.

Code:

```
#include<iostream>
using namespace std;

void merge(int arr[],int n,int brr[],int m){
   int i=n-1;
   int j=m-1;
   int k=m+n-1;
   while(i>=0 && j>=0){
      if(arr[i]>brr[j]){
         arr[k]=arr[i];
         i--;
         k--;
      }
      else{
         arr[k]=brr[j];
         j--;
         k--;
      }
   }
   while (j >= 0)
   {
      arr[k] = brr[j];
      j--;
      k--;
   }
}

int main(){
   int n,m;
   cin>>n>>m;
   int Arr[n];
   int Brr[m];
   for(int i=0;i<n;i++){
      cin>>Arr[i];
   }
   for(int i=0;i<m;i++){
      cin>>Brr[i];
   }
   merge(Arr,n,Brr,m);

   for(int i=0;i<n+m;i++)
      cout<<Arr[i]<<" ";

      return 0;

   }
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_1 at 11:36:31 AM
mkdir -p build && g++ 1_merge_sorted_arrays.cpp -o build/1_merge_sorted_arrays && ./build/1_merge_sorted_arrays
enter size of array a: 4
enter size of array b: 4
enter 4 sorted elements of array a: 1
2
3
4
enter 4 sorted elements of array b: 4
5
6
7
merged array: 1 2 3 4 4 5 6 7

Question 2: Group Anagrams

Problem Statement:

Write a method to sort an array of strings so that all the anagrams are next to each other.

Proposed Solution:

Two words are anagrams of each other if their characters, when sorted, produce the same string. This sorted string is used as a common key to group anagrams together.

An unordered_map is used where the key is the sorted version of each word and the value is a list of all original words that share that key.

Every word in the input array is processed once, its characters are sorted to generate the key, and the original word is inserted into the map against that key.

Finally, the map is traversed, and all words belonging to the same group (same key) are printed consecutively, placing all anagrams next to each other.

Time Complexity: O(n · k log k), where n is the number of words and k is the average length of a word, since each word is sorted individually.

Space Complexity: O(n · k), for storing all words and their sorted keys in the hash map.

Code:

```
#include <bits/stdc++.h>
using namespace std;
void groupAnagrams(vector<string>&arr){
   unordered_map<string,vector<string>>mp;
   for(string word:arr){
      string key=word;
      sort(key.begin(),key.end());
      mp[key].push_back(word);
   }
   for(auto group:mp){
      for(string word:group.second){
         cout<<word<<" ";
      }
   }
}
int main()
{
   vector<string>arr={"abc","cab","bca","xyz","zyx","pqr"};
   groupAnagrams(arr);
   return 0;
}
```

Output:

```
 Linux@atharvk  main in  ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_1
└─ mkdir -p build && g++ 2_group_anagrams.cpp -o build/2_group_anagrams && ./build/2_group_anagrams
enter number of words: 4
enter 4 words: atharv
shree
ram
sita
grouped anagrams: sita ram shree atharv %
```

Question 3: Search in a Rotated Sorted Array

Problem Statement:

Given a sorted array of n integers that has been rotated an unknown number of times, write code to find an element in the array. You may assume that the array was originally sorted in increasing order.

Proposed Solution:

A rotated sorted array can be viewed as two sorted subarrays joined together. At any point during binary search, at least one half (left of mid or right of mid) is guaranteed to be properly sorted.

At each step, the middle element is checked first. If it matches the target, its index is returned directly.

Otherwise, it is determined which half (low to mid, or mid to high) is sorted by comparing arr[low] with arr[mid]. If the target lies within the range of the sorted half, the search continues in that half; otherwise, the search moves to the other half.

This halves the search space in every iteration, similar to standard binary search, even though the array is rotated.

Time Complexity: O(log n), since the search space is reduced by half at every step, just like standard binary search.

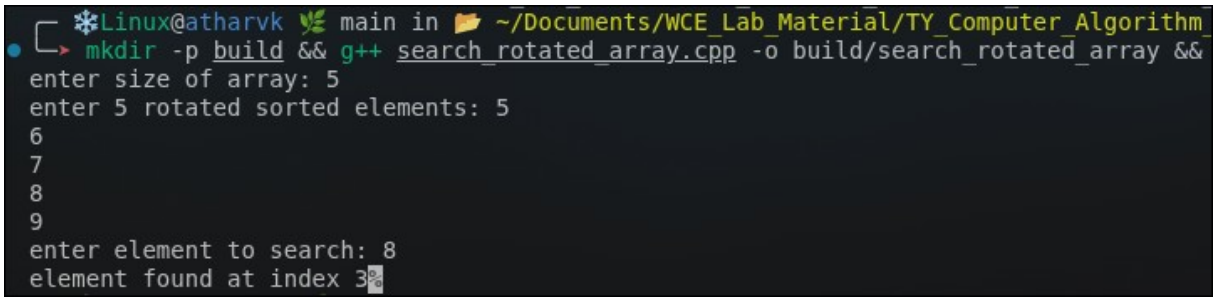
Space Complexity: O(1), as no extra data structures are used and the search is performed iteratively.

Code:

```
#include<iostream>
using namespace std;

int search(int arr[],int n,int target){
   int low=0,high=n-1;

   while(low<=high){
      int mid=(low+high)/2;

      if(arr[mid]==target)
         return mid;

      if(arr[low]<=arr[mid]){
         if(target>=arr[low]&&target<arr[mid])
            high=mid-1;
         else
            low=mid+1;
      }
      else{
         if(target>arr[mid]&&target<=arr[high])
            low=mid+1;
         else
```

```
            high=mid-1;
        }
    }
    return -1;
}

int main(){
    int n;
    cin>>n;
    int arr[n];

    for(int i=0;i<n;i++){
        cin>>arr[i];
    }
    cout<<search(arr,n,5);
}
```

Output:



Question 4: Sorting a 20GB File

Problem Statement:

Imagine you have a 20GB file with one string per line. Explain how you would sort the file.

Proposed Solution:

Since a 20GB file cannot be loaded entirely into main memory, an External Merge Sort approach is used, which works in two phases.

Phase 1 (Chunking and Sorting): The large file is divided into smaller chunks that comfortably fit into available RAM (for example, 100MB each). Each chunk is loaded, sorted in memory using an efficient in-memory sort, and written back to disk as a separate sorted temporary file.

Phase 2 (K-way Merging): All the sorted chunk files are then merged together using a k-way merge, typically implemented with a min-heap that always picks the smallest current element among all chunks. This produces the final fully sorted file without ever loading the whole dataset into memory at once.

This is a theoretical/design question, so no source code file has been implemented for it.

Time Complexity: O(n log n), where n is the total number of lines — O(n log k) for sorting chunks plus O(n log k) for the k-way merge, k being the number of chunks.

Space Complexity: *O(chunk size) in main memory at a time, and O(n) auxiliary space on disk for the temporary sorted chunk files.*

Question 5: Sparse Search

Problem Statement:

Given a sorted array of strings which is interspersed with empty strings, write a method to find the location of a given string. Example: find "ball" in {"at", "", "", "ball", "", "", "car", "", "", "dad", "", ""} → Output: 4

Proposed Solution:

The array is still sorted apart from the empty strings scattered within it, so a modified binary search is used instead of a linear scan.

At each step, if the element at mid is an empty string, mid is moved forward until a non-empty string is found (or the high boundary is crossed).

If no non-empty element is found on the right side of the range, the search range is shrunk from the high end and the search continues from a recalculated midpoint.

Once a valid non-empty mid element is found, its value is compared with the target and the search range is narrowed left or right exactly as in standard binary search.

Time Complexity: O(log n) on average; worst case degrades to O(n) when the array contains long runs of consecutive empty strings.

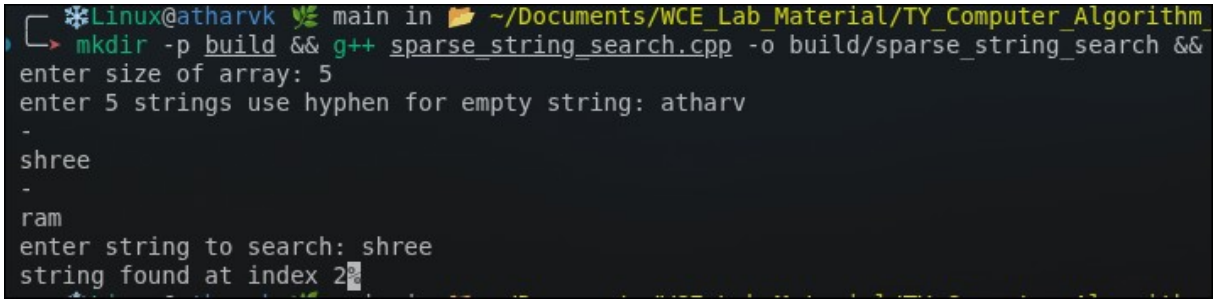
Space Complexity: O(1), since the search is performed iteratively without additional data structures.

Code:

```
#include<iostream>
#include<vector>
using namespace std;

int search(vector<string> &arr,string target){
   int low=0,high=arr.size()-1;

   while(low<=high){
      int mid=(low+high)/2;

      while(mid<=high&&arr[mid]=="")
         mid++;

      if(mid>high){
         high=(low+high)/2-1;
         continue;
      }

      if(arr[mid]==target)
         return mid;

      if(arr[mid]<target)
         low=mid+1;
      else
         high=mid-1;
   }
```

```
    return -1;
}

int main(){
    vector<string> arr={"at","","","ball","","","car","","dad","",""};

    cout<<search(arr,"ball");
}
```

Output:



Question 6: Search in a Sorted Matrix

Problem Statement:

Given an M×N matrix in which each row and each column is sorted in ascending order, write a method to find an element.

Proposed Solution:

The search starts from the top-right corner of the matrix, a position that is the largest element in its row and the smallest element in its column, which allows discarding a full row or column at each step.

If the current element equals the target, its position is returned immediately.

If the current element is greater than the target, the entire current column can be eliminated by moving one step left, since all elements below in that column are even larger.

If the current element is smaller than the target, the entire current row can be eliminated by moving one step down, since all elements to the left in that row are even smaller.

This continues until the target is found or the search pointers move outside the matrix bounds.

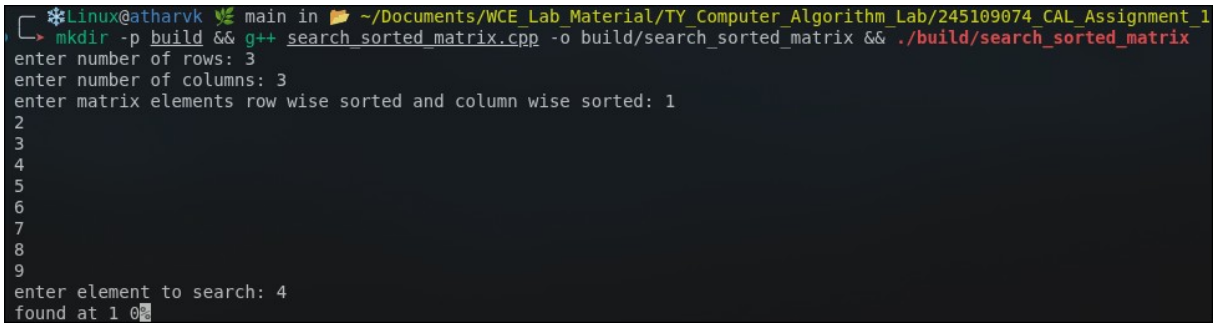
Time Complexity: O(m + n), where m and n are the number of rows and columns, since each step eliminates one row or one column.

Space Complexity: O(1), as the search uses only two index variables and no extra storage.

Code:

```
#include<iostream>
using namespace std;

int main(){
   int arr[4][4]={
      {1,4,7,11},
      {2,5,8,12},
      {3,6,9,16},
      {10,13,14,17}
   };
   int target=9;
   int i=0,j=3;
   while(i<4&&j>=0){
      if(arr[i][j]==target){
         cout<<"Found at "<<i<<" "<<j;
         return 0;
      }
      if(arr[i][j]>target)
         j--;
      else
         i++;
   }

   cout<<"Not Found";
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_1
mkdir -p build && g++ search_sorted_matrix.cpp -o build/search_sorted_matrix && ./build/search_sorted_matrix
enter number of rows: 3
enter number of columns: 3
enter matrix elements row wise sorted and column wise sorted: 1
2
3
4
5
6
7
8
9
enter element to search: 4
found at 1 0

Question 7: Circus Tower

Problem Statement:

A circus is designing a tower routine consisting of people standing atop one another's shoulders. Each person must be both shorter and lighter than the person below him or her. Given the heights and weights of each person, write a method to compute the largest possible number of people in such a tower.

Proposed Solution:

All the people are first sorted primarily by height, and by weight as a tie-breaker, so that height increases (or stays consistent) while scanning through the sorted list from the first to the last person.

Starting from the first (shortest) person in the sorted order, each subsequent person is checked — if their weight is strictly greater than the weight of the last person added to the tower, they are added to the tower list.

This greedily builds an increasing sequence of height-weight pairs while scanning the sorted array a single time, and the final count of people selected gives the tower size.

The heights and weights of the selected people are printed from top (shortest/lightest) to bottom (tallest/heaviest) of the tower.

Time Complexity: O(n log n), dominated by the initial sort of n people; the subsequent single scan takes O(n).

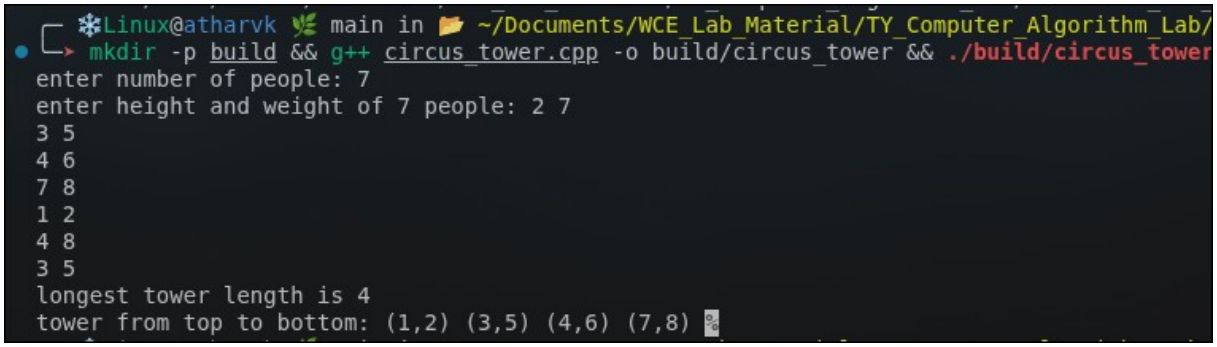
Space Complexity: O(n), for storing the list of people and the resulting tower.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Person {
   int height;
   int weight;
};

bool compare(Person a, Person b) {
   if (a.height == b.height)
      return a.weight < b.weight;
   return a.height < b.height;
}

int main() {
   int n;
   cout<<"Enter number of people: ";
   cin>>n;

   vector<Person> people(n);

   cout<<"Enter height and weight:\n";
   for (int i=0; i < n; i++)
```

```
    cin>>people[i].height>>people[i].weight;

  sort(people.begin(), people.end(), compare);

  cout<<"\nTower (Top to Bottom):\n";

  int count=1;
  cout<<"("<<people[0].height<<", "<<people[0].weight<<")\n";

  for (int i=1; i < n; i++) {
    if (people[i].weight > people[i - 1].weight) {
      cout<<"("<<people[i].height<<", "<<people[i].weight<<")\n";
      count++;
    }
  }

  cout<<"\nLongest Tower Length="<<count<<endl;

  return 0;
}
```

Output:



Question 8: Rank of a Stream

Problem Statement:

Imagine you are reading in a stream of integers. Periodically, you wish to look up the rank of a number x (the number of values less than x seen so far). Implement track(int x), called whenever a new number is generated, and getRankOfNumber(int x), which returns the number of values less than x.

Proposed Solution:

A class RankStream is maintained with a vector that stores every number as it arrives via the track() method.

The track() method simply appends the incoming number to the internal vector in O(1) time, keeping the stream of numbers in the order they were received.

The getRankOfNumber() method iterates through all stored numbers and counts how many of them are strictly less than the queried value x, returning this count as the rank.

Time Complexity: O(1) for track(); O(n) for getRankOfNumber(), where n is the number of elements streamed so far.

Space Complexity: O(n), for storing all the streamed numbers in the vector.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class RankStream {
   vector<int> nums;
public:
   void track(int x) {
      nums.push_back(x);
   }
   int getRankOfNumber(int x) {
      int count=0;
      for (int num : nums) {
         if (num<x)
            count++;
      }
      return count;
   }
};

int main() {
   RankStream rs;
   int n;
   cout<<"Enter number of elements in stream: ";
   cin>>n;
   cout<<"Enter stream elements:\n";
   for (int i=0; i<n; i++) {
```

```
        int x;
        cin>>x;
        rs.track(x);
    }
    int q;
    cout<<"Enter number to find rank: ";
    cin>>q;
    cout<<"Rank of "<<q<<"="<<rs.getRankOfNumber(q)<<endl;
    return 0;
}
```

Output: